

MSDS422 Assignment 4: Neural Networks

```
In [1]: import pandas as pd
import numpy as np
import math
from operator import itemgetter
import tensorflow as tf

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
import sklearn.metrics as metrics

from sklearn import tree
from sklearn.tree import _tree

from sklearn.preprocessing import MinMaxScaler

from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import RandomForestClassifier

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from mlxtend.plotting import plot SequentialFeatureSelection as plot_sfs

import warnings
warnings.filterwarnings("ignore")

sns.set()

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
```

Load In Dataset

```
In [2]: file = "C:\\Users\\Ivan\\Documents\\MSDS422\\Assignment 1\\HMEQ_Loss.csv"

df = pd.read_csv( file )

TARGET_F = "TARGET_BAD_FLAG"
TARGET_A = "TARGET_LOSS_AMT"
```

```
dt = df.dtypes
```

```
In [3]: objList = []
numList = []
for i in dt.index :
    if i in ( [ TARGET_F, TARGET_A ] ) : continue
    if dt[i] in (["object"]) : objList.append( i )
    if dt[i] in (["float64","int64"]) : numList.append( i )
```

Deal with Missing Values

```
In [4]: for i in objList :
    if df[i].isna().sum() == 0 : continue
    NAME = "IMP_" + i
    df[NAME] = df[i]
    df[NAME] = df[NAME].fillna("MISSING")
    g = df.groupby( NAME )
    df = df.drop( i, axis=1 )

dt = df.dtypes
objList = []
for i in dt.index :
    if i in ( [ TARGET_F, TARGET_A ] ) : continue
    if dt[i] in (["object"]) : objList.append( i )
```

```
In [5]: dt = df.dtypes
objList = []
for i in dt.index :
    if i in ( [ TARGET_F, TARGET_A ] ) : continue
    if dt[i] in (["object"]) : objList.append( i )

for i in objList :
    thePrefix = "z_" + i
    y = pd.get_dummies( df[i], prefix=thePrefix, drop_first=True )
    df = pd.concat( [df, y], axis=1 )
```

```
In [6]: print(df.columns.tolist())
```

```
['TARGET_BAD_FLAG', 'TARGET_LOSS_AMT', 'LOAN', 'MORTDUE', 'VALUE', 'YOJ', 'DEROG',
'DELINQ', 'CLAGE', 'NINQ', 'CLNO', 'DEBTINC', 'IMP_REASON', 'IMP_JOB', 'z_IMP_REASON
_HomeImp', 'z_IMP_REASON_MISSING', 'z_IMP_JOB_Mgr', 'z_IMP_JOB_Office', 'z_IMP_JOB_O
ther', 'z_IMP_JOB_ProfExe', 'z_IMP_JOB_Sales', 'z_IMP_JOB_Self']
```

```
In [7]: for i in numList:
    FLAG = "M_" + i
    IMP = "IMP_" + i
    df[ FLAG ] = df[i].isna() + 0
    df[ IMP ] = df[ i ]
    df.loc[ df[IMP].isna(), IMP ] = df[i].median()
    df = df.drop( i, axis=1 )
```

Deal with Outliers

```
In [8]: dt = df.dtypes
numList = []
for i in dt.index :
    if i in ( [ TARGET_F, TARGET_A ] ) : continue
    if dt[i] in (["float64","int64"]) : numList.append( i )

for i in numList :
    theMean = df[i].mean()
    theSD = df[i].std()
    theMax = df[i].max()
    theCutoff = round( theMean + 3*theSD )
    if theMax < theCutoff : continue
    FLAG = "O_" + i
    TRUNC = "TRUNC_" + i
    df[ FLAG ] = ( df[i] > theCutoff )+ 0
    df[ TRUNC ] = df[ i ]
    df.loc[ df[TRUNC] > theCutoff, TRUNC ] = theCutoff
    df = df.drop( i, axis=1 )
```

```
In [9]: for i in objList:
        df = df.drop( i, axis=1 )
```

Create Testing and Training Datasets

```
In [10]: X = df.copy()
X = X.drop( TARGET_F, axis=1 )
X = X.drop( TARGET_A, axis=1 )

Y = df[[TARGET_F, TARGET_A]]

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, train_size=0.8, test_size=0.2)

F = ~ Y_train[ TARGET_A ].isna()
W_train = X_train[F].copy()
Z_train = Y_train[F].copy()

F = ~ Y_test[ TARGET_A ].isna()
W_test = X_test[F].copy()
Z_test = Y_test[F].copy()

F = Z_train[ TARGET_A ] > 25000
Z_train.loc[ F, TARGET_A ] = 25000

F = Z_test[ TARGET_A ] > 25000
Z_test.loc[ F, [TARGET_A] ] = 25000
```

Model Accuracy Functions

```

In [11]: def getProbAccuracyScores( NAME, MODEL, X, Y ) :
    pred = MODEL.predict( X )
    probs = MODEL.predict_proba( X )
    acc_score = metrics.accuracy_score(Y, pred)
    p1 = probs[:,1]
    fpr, tpr, threshold = metrics.roc_curve( Y, p1)
    auc = metrics.auc(fpr,tpr)
    return [NAME, acc_score, fpr, tpr, auc]

def print_ROC_Curve( TITLE, LIST ) :
    fig = plt.figure(figsize=(6,4))
    plt.title( TITLE )
    for theResults in LIST :
        NAME = theResults[0]
        fpr = theResults[2]
        tpr = theResults[3]
        auc = theResults[4]
        theLabel = "AUC " + NAME + ' %0.2f' % auc
        plt.plot(fpr, tpr, label = theLabel )
    plt.legend(loc = 'lower right')
    plt.plot([0, 1], [0, 1], 'r--')
    plt.xlim([0, 1])
    plt.ylim([0, 1])
    plt.ylabel('True Positive Rate')
    plt.xlabel('False Positive Rate')
    plt.show()

def print_Accuracy( TITLE, LIST ) :
    print( TITLE )
    print( "=====" )
    for theResults in LIST :
        NAME = theResults[0]
        ACC = theResults[1]
        print( NAME, " = ", ACC )
    print( "-----\n\n" )

def getAmtAccuracyScores( NAME, MODEL, X, Y ) :
    pred = MODEL.predict( X )
    MEAN = Y.mean()
    RMSE = math.sqrt( metrics.mean_squared_error( Y, pred))
    return [NAME, RMSE, MEAN]

def getCoefLogit( MODEL, TRAIN_DATA ) :
    varNames = list( TRAIN_DATA.columns.values )
    coef_dict = {}
    coef_dict["INTERCEPT"] = MODEL.intercept_[0]
    for coef, feat in zip(MODEL.coef_[0],varNames):
        coef_dict[feat] = coef
    print("\nDEFAULT")
    print("-----")
    print("Total Variables: ", len( coef_dict ) )
    for i in coef_dict :
        print( i, " = ", coef_dict[i] )

def getCoefLinear( MODEL, TRAIN_DATA ) :

```



```

varNames = list( TRAIN_DATA.columns.values )
coef_dict = {}
coef_dict["INTERCEPT"] = MODEL.intercept_
for coef, feat in zip(MODEL.coef_, varNames):
    coef_dict[feat] = coef
print("\nLOSSES")
print("-----")
print("Total Variables: ", len( coef_dict ) )
for i in coef_dict :
    print( i, " = ", coef_dict[i] )

def get_TF_ProbAccuracyScores( NAME, MODEL, X, Y):
    probs = MODEL.predict(X)
    pred_list = []
    for p in probs:
        pred_list.append(np.argmax(p))
    pred = np.array(pred_list)
    acc_score = metrics.accuracy_score(Y, pred)
    p1 = probs[:,1]
    fpr, tpr, threshold = metrics.roc_curve(Y, p1)
    auc = metrics.auc(fpr, tpr)
    return [NAME, acc_score, fpr, tpr, auc]

```

Decision Tree: Variables and Regression Models

```

In [12]: def getTreeVars( TREE, varNames ) :
    tree_ = TREE.tree_
    varName = [ varNames[i] if i != _tree.TREE_UNDEFINED else "undefined!" for i in

    nameSet = set()
    for i in tree_.feature :
        if i != _tree.TREE_UNDEFINED :
            nameSet.add( i )
    nameList = list( nameSet )
    parameter_list = list()
    for i in nameList :
        parameter_list.append( varNames[i] )
    return parameter_list

```

Loan Default Probability

```

In [13]: WHO = "TREE"

BAD = tree.DecisionTreeClassifier( max_depth=4 )
BAD = BAD.fit( X_train, Y_train[ TARGET_F ] )

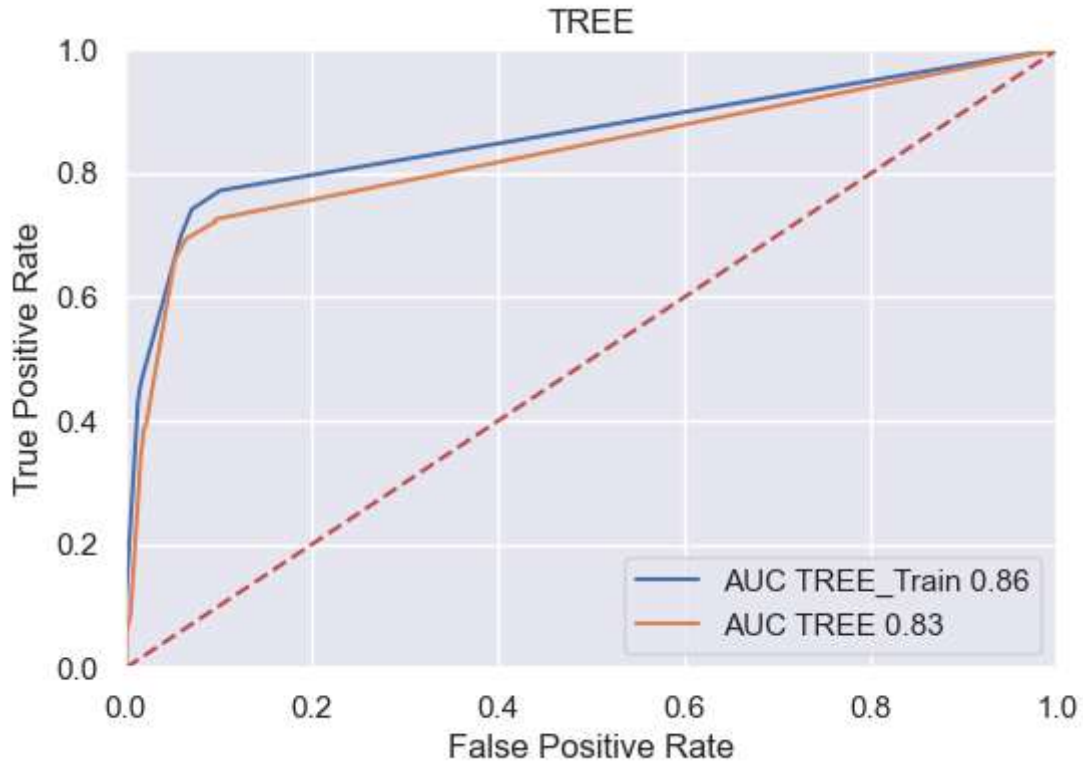
TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train, Y_train[ TARGET_F ]
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test, Y_test[ TARGET_F ] )

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )

feature_cols = list( X.columns.values )

```

```
tree.export_graphviz(BAD, out_file='tree_f.txt',filled=True, rounded=True, feature_
vars_tree_flag = getTreeVars( BAD, feature_cols )
```



```
TREE CLASSIFICATION ACCURACY
=====
TREE_Train = 0.8928271812080537
TREE = 0.8875838926174496
-----
```

Losses

```
In [14]: AMT = tree.DecisionTreeRegressor( max_depth= 4 )
AMT = AMT.fit( W_train, Z_train[TARGET_A] )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train, Z_train[TARGET_A] )
TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test, Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

feature_cols = list( X.columns.values )
vars_tree_amt = getTreeVars( AMT, feature_cols )
tree.export_graphviz(AMT,out_file='tree_a.txt',filled=True, rounded=True, feature_n

TREE_BAD = TEST_BAD.copy()
TREE_AMT = TEST_AMT.copy()
```

```

TREE  RMSE  ACCURACY
=====
TREE_Train  =  3216.216446881694
TREE  =  3942.9974837311943
-----

```

Random Forest

```

In [15]: def getEnsembleTreeVars( ENSTREE, varNames ) :
            importance = ENSTREE.feature_importances_
            index = np.argsort(importance)
            theList = []
            for i in index :
                imp_val = importance[i]
                if imp_val > np.average( ENSTREE.feature_importances_ ) :
                    v = int( imp_val / np.max( ENSTREE.feature_importances_ ) * 100 )
                    theList.append( ( varNames[i], v ) )
            theList = sorted(theList,key=itemgetter(1),reverse=True)
            return theList

```

```

In [16]: WHO = "RF"

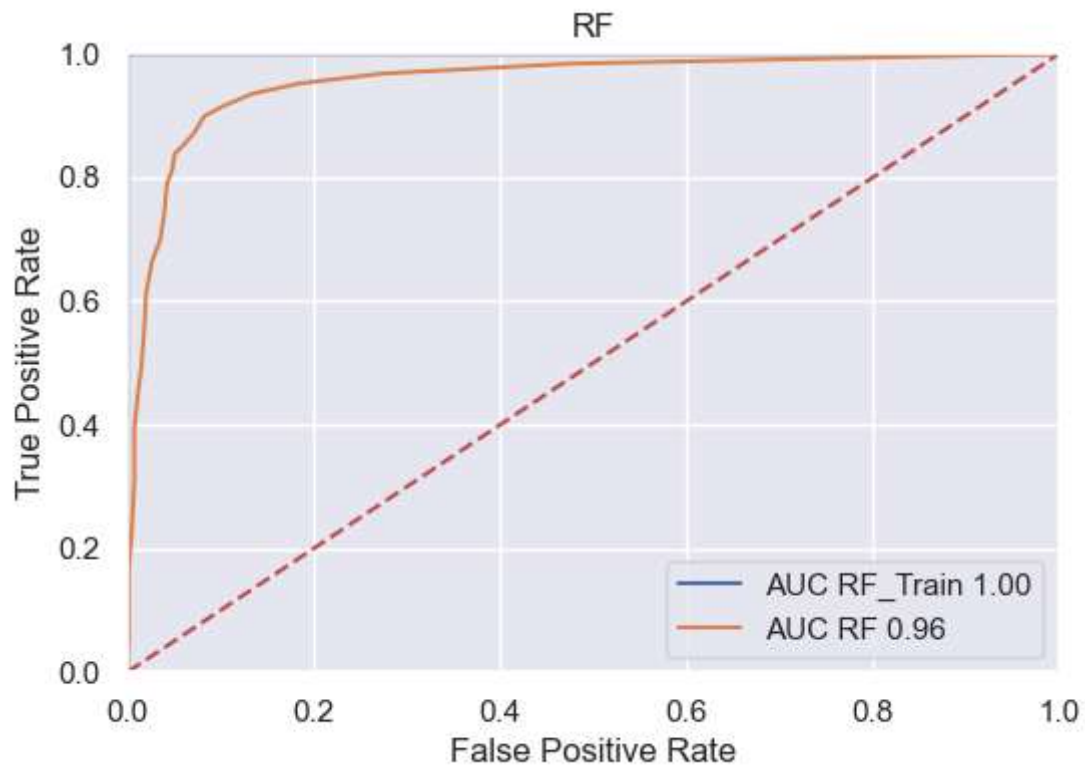
BAD = RandomForestClassifier( n_estimators = 25, random_state=1 )
BAD = BAD.fit( X_train, Y_train[ TARGET_F ] )

TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train, Y_train[ TARGET_F ] )
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test, Y_test[ TARGET_F ] )

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )

feature_cols = list( X.columns.values )
vars_RF_flag = getEnsembleTreeVars( BAD, feature_cols )

```



RF CLASSIFICATION ACCURACY

=====

RF_Train = 0.9995805369127517

RF = 0.9093959731543624

Loss

```
In [17]: AMT = RandomForestRegressor(n_estimators = 100, random_state=1)
AMT = AMT.fit( W_train, Z_train[TARGET_A] )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train, Z_train[TARGET_A] )
TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test, Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

feature_cols = list( X.columns.values )
vars_RF_amt = getEnsembleTreeVars( AMT, feature_cols )

RF_BAD = TEST_BAD.copy()
RF_AMT = TEST_AMT.copy()
```

RF RMSE ACCURACY

=====

RF_Train = 833.5292534358123

RF = 2178.4972076593153

Gradient Boosting

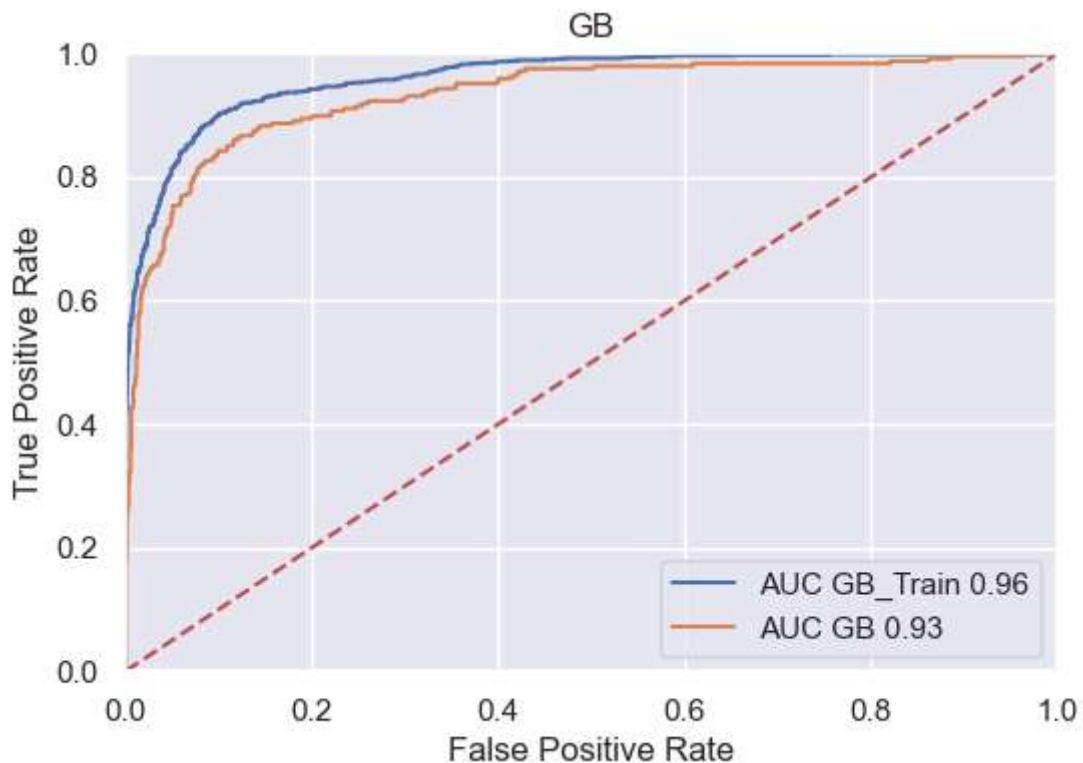
```
In [18]: WHO = "GB"

BAD = GradientBoostingClassifier( random_state=1 )
BAD = BAD.fit( X_train, Y_train[ TARGET_F ] )

TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train, Y_train[ TARGET_F ] )
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test, Y_test[ TARGET_F ] )

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )

feature_cols = list( X.columns.values )
vars_GB_flag = getEnsembleTreeVars( BAD, feature_cols )
```



```
GB CLASSIFICATION ACCURACY
=====
GB_Train = 0.9238674496644296
GB       = 0.9068791946308725
-----
```

Losses

```
In [19]: AMT = GradientBoostingRegressor(random_state=1)
AMT = AMT.fit( W_train, Z_train[TARGET_A] )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train, Z_train[TARGET_A] )
```

```

TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test, Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

feature_cols = list( X.columns.values )
vars_GB_amt = getEnsembleTreeVars( AMT, feature_cols )

GB_BAD = TEST_BAD.copy()
GB_AMT = TEST_AMT.copy()

```

GB RMSE ACCURACY

=====

GB_Train = 1056.527956864017

GB = 1872.5893967074198

Regression: All Variables

In [20]: WHO = "REG_ALL"

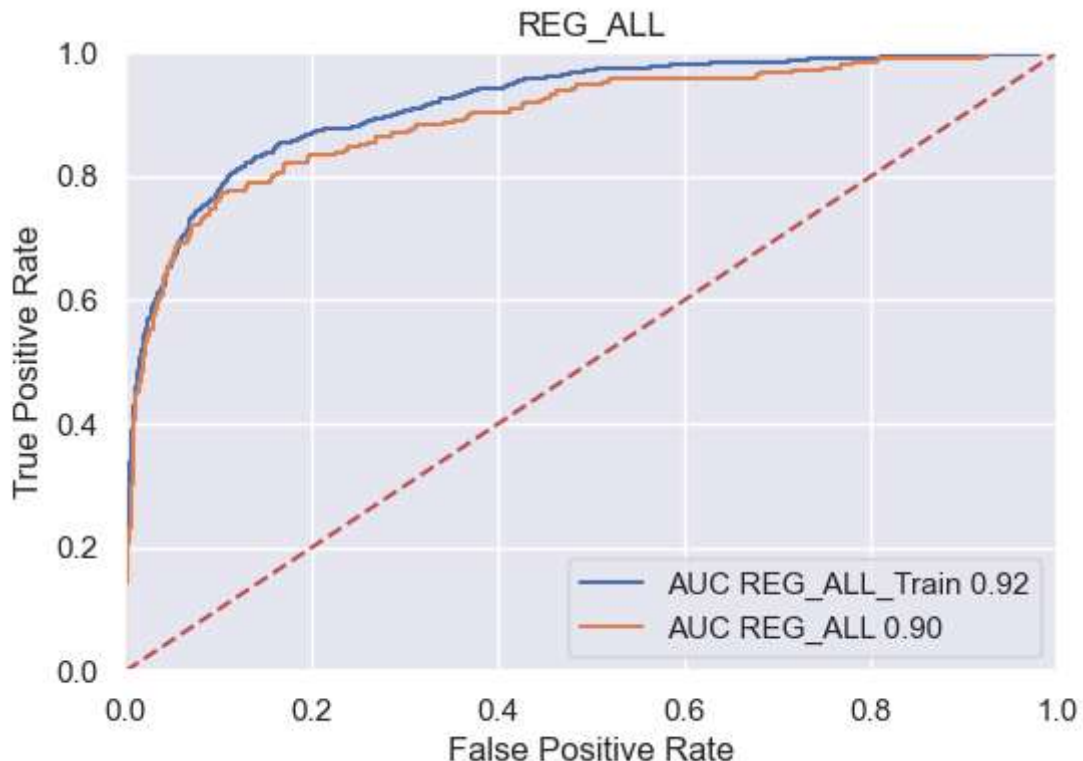
```

BAD = LogisticRegression( solver='newton-cg', max_iter=1000 )
BAD = BAD.fit( X_train, Y_train[ TARGET_F ] )

TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train, Y_train[ TARGET_F ] )
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test, Y_test[ TARGET_F ] )

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )

```



```
REG_ALL CLASSIFICATION ACCURACY
=====
REG_ALL_Train = 0.893246644295302
REG_ALL = 0.886744966442953
-----
```

Losses

```
In [21]: AMT = LinearRegression()
          AMT = AMT.fit( W_train, Z_train[TARGET_A] )

          TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train, Z_train[TARGET_A] )
          TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test, Z_test[TARGET_A] )
          print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )
```

```
REG_ALL RMSE ACCURACY
=====
REG_ALL_Train = 2810.5833219983
REG_ALL = 2936.120283227421
-----
```

```
In [22]: varNames = list( X_train.columns.values )

          REG_ALL_BAD_COEF = getCoefLogit( BAD, X_train )
          REG_ALL_AMT_COEF = getCoefLinear( AMT, X_train )

          REG_ALL_BAD = TEST_BAD.copy()
          REG_ALL_AMT = TEST_AMT.copy()
```

DEFAULT

Total Variables: 39
INTERCEPT = -5.28382948220677
z_IMP_REASON_HomeImp = 0.18241468422617138
z_IMP_REASON_MISSING = -0.15098431268842014
z_IMP_JOB_Mgr = 0.9090541090919912
z_IMP_JOB_Office = 0.2779864014569086
z_IMP_JOB_Other = 0.9646758107805806
z_IMP_JOB_ProfExe = 0.6698418249525967
z_IMP_JOB_Sales = 1.9957744054219546
z_IMP_JOB_Self = 1.0299796308139835
M_LOAN = 0.0
M_MORTDUE = 0.16166485331313385
M_VALUE = 3.7280441498597394
M_YOJ = -0.6282978849707822
M_DEROG = -1.7609646015723928
M_DELINQ = -0.17133725838932545
M_CLAGE = 1.0504049431233244
M_NINQ = -0.16874791159520122
M_CLNO = 2.0861089485604287
M_DEBTINC = 2.6407934611371044
O_IMP_LOAN = -0.19176403970712064
TRUNC_IMP_LOAN = -2.0055287225130596e-06
O_IMP_MORTDUE = -0.11425170923591876
TRUNC_IMP_MORTDUE = -1.3169573851231086e-06
O_IMP_VALUE = 1.012720732226957
TRUNC_IMP_VALUE = 3.7125168505827964e-07
O_IMP_YOJ = 0.6979288402918274
TRUNC_IMP_YOJ = -0.0158893805767944
O_IMP_DEROG = 0.5125117893623137
TRUNC_IMP_DEROG = 0.5764459303695432
O_IMP_DELINQ = 1.8548455597125049
TRUNC_IMP_DELINQ = 0.7519677133509852
O_IMP_CLAGE = 0.7518860474639155
TRUNC_IMP_CLAGE = -0.006252900175874569
O_IMP_NINQ = -0.14150024060414015
TRUNC_IMP_NINQ = 0.17355180087605496
O_IMP_CLNO = 0.8416440459587182
TRUNC_IMP_CLNO = -0.014625865500131903
O_IMP_DEBTINC = 2.712029579383601
TRUNC_IMP_DEBTINC = 0.08192733996338301

LOSSES

Total Variables: 39
INTERCEPT = -6514.0795763533115
z_IMP_REASON_HomeImp = -2375.15294104569
z_IMP_REASON_MISSING = -709.3894703095692
z_IMP_JOB_Mgr = -913.3293943701728
z_IMP_JOB_Office = -1309.8533992981081
z_IMP_JOB_Other = -989.5244615941473
z_IMP_JOB_ProfExe = -814.1042885339051
z_IMP_JOB_Sales = 837.4040095853227
z_IMP_JOB_Self = -208.8907777231918
M_LOAN = -1.028865881380625e-11


```

M_MORTDUE = -371.19753815317955
M_VALUE = 289.2119937361625
M_YOJ = -0.6240220984530574
M_DEROG = 359.0951302890346
M_DELIHQ = 1536.0279029276267
M_CLAGE = -3892.5129894182437
M_NINQ = -611.2129144487695
M_CLNO = 4532.516643773778
M_DEBTINC = 4191.427274463983
O_IMP_LOAN = -4216.375683297988
TRUNC_IMP_LOAN = 0.5237306454365167
O_IMP_MORTDUE = -2832.906391678072
TRUNC_IMP_MORTDUE = 0.007522954769237498
O_IMP_VALUE = -366.2913692178466
TRUNC_IMP_VALUE = -0.0030579868988459507
O_IMP_YOJ = -3107.811857596906
TRUNC_IMP_YOJ = -56.532389001227045
O_IMP_DEROG = 282.7152328438487
TRUNC_IMP_DEROG = 473.46083609333886
O_IMP_DELIHQ = 1323.4873031676395
TRUNC_IMP_DELIHQ = 733.0250406795474
O_IMP_CLAGE = -1372.5273287753507
TRUNC_IMP_CLAGE = -19.188311169170557
O_IMP_NINQ = 397.74150730993875
TRUNC_IMP_NINQ = -19.43375948637048
O_IMP_CLNO = -890.5215267452435
TRUNC_IMP_CLNO = 178.64678119241205
O_IMP_DEBTINC = -1464.6250342126975
TRUNC_IMP_DEBTINC = 200.36408344837326

```

Regression: Decision Tree Variables

```

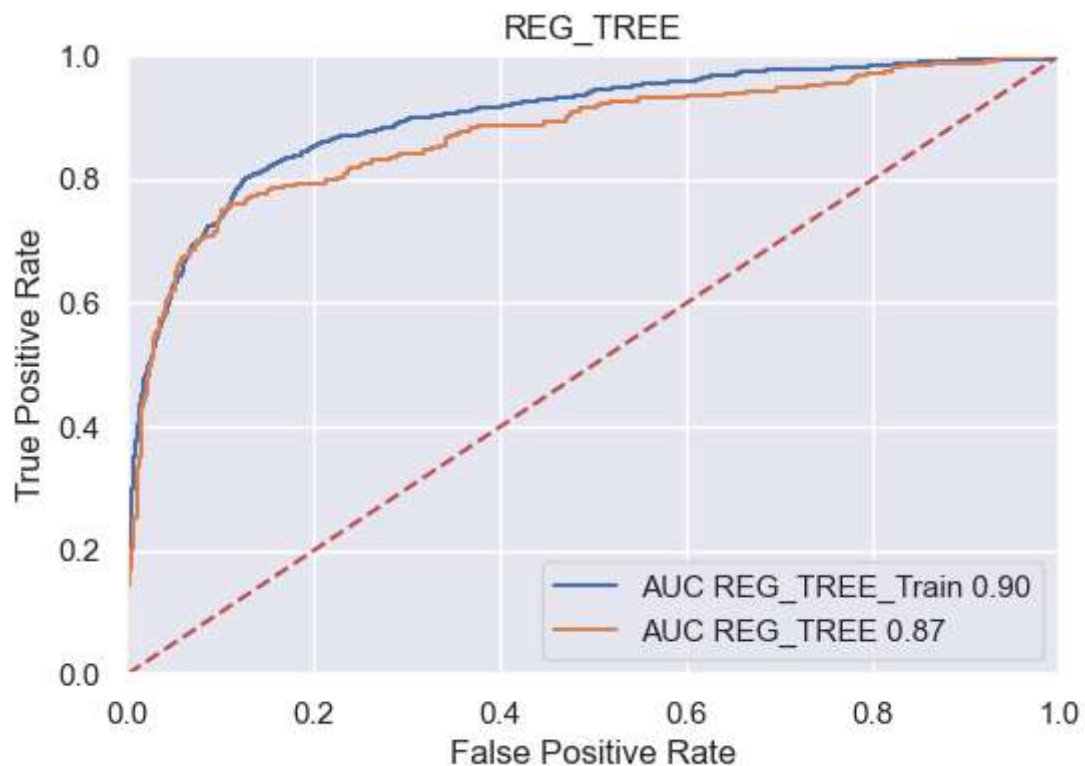
In [23]: WHO = "REG_TREE"

BAD = LogisticRegression( solver='newton-cg', max_iter=1000 )
BAD = BAD.fit( X_train[vars_tree_flag], Y_train[ TARGET_F ] )

TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train[vars_tree_flag], Y_
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test[vars_tree_flag], Y_test[ TARGET_

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )

```



```
REG_TREE CLASSIFICATION ACCURACY
=====
REG_TREE_Train = 0.8861157718120806
REG_TREE      = 0.8850671140939598
-----
```

Losses

```
In [24]: AMT = LinearRegression()
AMT = AMT.fit( W_train[vars_tree_amt], Z_train[TARGET_A] )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train[vars_tree_amt], Z_train[TARGET_A] )
TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test[vars_tree_amt], Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

varNames = list( X_train.columns.values )

REG_TREE_BAD_COEF = getCoefLogit( BAD, X_train[vars_tree_flag] )
REG_TREE_AMT_COEF = getCoefLinear( AMT, X_train[vars_tree_amt] )

REG_TREE_BAD = TEST_BAD.copy()
REG_TREE_AMT = TEST_AMT.copy()
```

```

REG_TREE RMSE ACCURACY
=====
REG_TREE_Train = 3729.7451449038385
REG_TREE = 3680.9694661223716
-----

DEFAULT
-----
Total Variables: 12
INTERCEPT = -5.2025895412174545
TRUNC_IMP_DEBTINC = 0.10465476962342855
M_VALUE = 3.7418670544564767
M_DEROG = -0.8036803571431179
M_DEBTINC = 2.7358483963235076
TRUNC_IMP_LOAN = -4.968121720548912e-06
TRUNC_IMP_VALUE = -3.800203083579819e-07
TRUNC_IMP_YOJ = -0.015280026888954145
TRUNC_IMP_DEROG = 0.6875087805092716
O_IMP_DELINQ = 2.1051181430849057
TRUNC_IMP_DELINQ = 0.6183144537388855
TRUNC_IMP_CLAGE = -0.006538797614733332

LOSSES
-----
Total Variables: 6
INTERCEPT = -10314.186170172457
TRUNC_IMP_CLNO = 216.8574568294074
TRUNC_IMP_DEBTINC = 186.0314985912876
M_DEBTINC = 4378.542674593453
TRUNC_IMP_LOAN = 0.5033651534424095
TRUNC_IMP_MORTDUE = -0.0016995084625364143

```

Regression: Random Forest Variables

```

In [25]: WHO = "REG_RF"

print("\n\n")
RF_flag = []
for i in vars_RF_flag :
    print(i)
    theVar = i[0]
    RF_flag.append( theVar )

print("\n\n")
RF_amt = []
for i in vars_RF_amt :
    print(i)
    theVar = i[0]
    RF_amt.append( theVar )

```

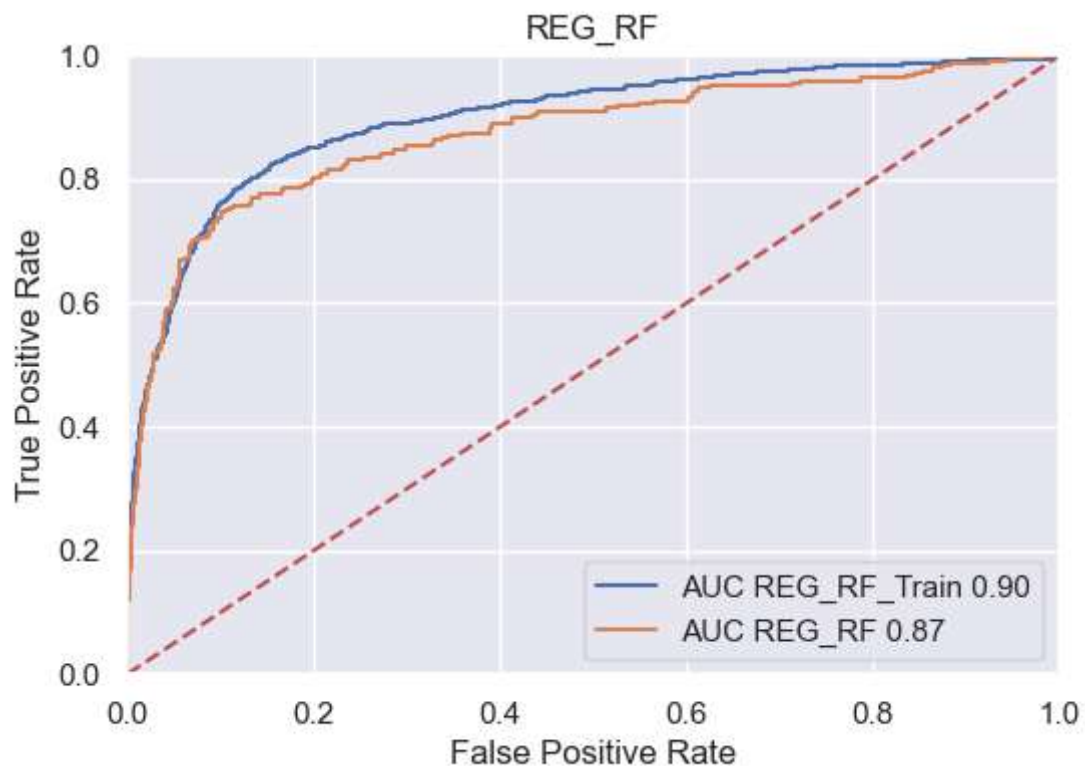
```
( 'M_DEBTINC', 100)
( 'TRUNC_IMP_DEBTINC', 67)
( 'TRUNC_IMP_CLAGE', 44)
( 'TRUNC_IMP_DELIQ', 42)
( 'TRUNC_IMP_LOAN', 40)
( 'TRUNC_IMP_MORTDUE', 36)
( 'TRUNC_IMP_VALUE', 36)
( 'TRUNC_IMP_CLNO', 35)
( 'TRUNC_IMP_YOJ', 28)
( 'TRUNC_IMP_DEROG', 24)
( 'TRUNC_IMP_NINQ', 20)
( 'M_VALUE', 16)
```

```
( 'TRUNC_IMP_LOAN', 100)
( 'TRUNC_IMP_CLNO', 13)
( 'TRUNC_IMP_DEBTINC', 10)
( 'M_DEBTINC', 4)
( 'TRUNC_IMP_CLAGE', 4)
```

```
In [26]: BAD = LogisticRegression( solver='newton-cg', max_iter=1000 )
BAD = BAD.fit( X_train[RF_flag], Y_train[ TARGET_F ] )

TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train[RF_flag], Y_train[
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test[RF_flag], Y_test[ TARGET_F ] )

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )
```



```
REG_RF CLASSIFICATION ACCURACY
=====
REG_RF_Train = 0.8800335570469798
REG_RF      = 0.87751677852349
-----
```

Losses

```
In [27]: AMT = LinearRegression()
AMT = AMT.fit( W_train[RF_amt], Z_train[TARGET_A] )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train[RF_amt], Z_train[TAR
TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test[RF_amt], Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

REG_RF_BAD_COEF = getCoefLogit( BAD, X_train[RF_flag] )
REG_RF_BAD_COEF = getCoefLinear( AMT, X_train[RF_amt] )

REG_RF_BAD = TEST_BAD.copy()
REG_RF_AMT = TEST_AMT.copy()
```

```

REG_RF RMSE ACCURACY
=====
REG_RF_Train = 3477.5386484194455
REG_RF = 3484.9713909591555
-----

DEFAULT
-----
Total Variables: 13
INTERCEPT = -5.247777884402919
M_DEBTINC = 2.6987099523612574
TRUNC_IMP_DEBTINC = 0.1026745279701456
TRUNC_IMP_CLAGE = -0.005891785747019949
TRUNC_IMP_DELINQ = 0.7105199977532758
TRUNC_IMP_LOAN = -5.028873402521499e-06
TRUNC_IMP_MORTDUE = -1.1491018663562309e-06
TRUNC_IMP_VALUE = 1.5653520638901784e-06
TRUNC_IMP_CLNO = -0.018774911357839708
TRUNC_IMP_YOJ = -0.011920792116538357
TRUNC_IMP_DEROG = 0.7179995237507223
TRUNC_IMP_NINQ = 0.13273645827782288
M_VALUE = 3.55206406272452

LOSSES
-----
Total Variables: 6
INTERCEPT = -7395.372854525076
TRUNC_IMP_LOAN = 0.5160302371830359
TRUNC_IMP_CLNO = 252.31239889601392
TRUNC_IMP_DEBTINC = 158.01870021930364
M_DEBTINC = 4402.276041744145
TRUNC_IMP_CLAGE = -19.944120943066338

```

Regression: Gradient Boosting Variables

```

In [28]: WHO = "REG_GB"

print("\n\n")
GB_flag = []
for i in vars_GB_flag :
    print(i)
    theVar = i[0]
    GB_flag.append( theVar )

print("\n\n")
GB_amt = []
for i in vars_GB_amt :
    print(i)
    theVar = i[0]
    GB_amt.append( theVar )

```

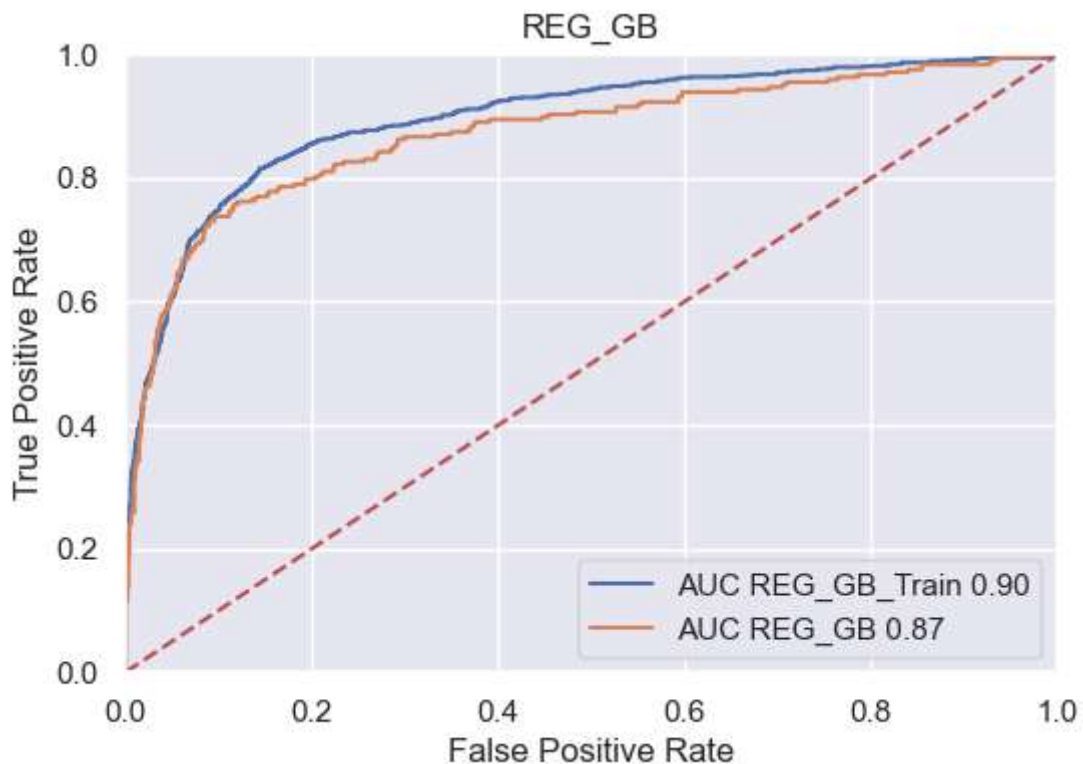
```
( 'M_DEBTINC', 100)
( 'TRUNC_IMP_DEBTINC', 29)
( 'TRUNC_IMP_DELIQ', 16)
( 'TRUNC_IMP_CLAGE', 14)
( 'TRUNC_IMP_DEROG', 7)
( 'M_VALUE', 7)
( 'TRUNC_IMP_CLNO', 5)
```

```
( 'TRUNC_IMP_LOAN', 100)
( 'TRUNC_IMP_CLNO', 16)
( 'TRUNC_IMP_DEBTINC', 8)
( 'M_DEBTINC', 7)
( 'TRUNC_IMP_CLAGE', 4)
```

```
In [29]: BAD = LogisticRegression( solver='newton-cg', max_iter=1000 )
BAD = BAD.fit( X_train[GB_flag], Y_train[ TARGET_F ] )

TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, X_train[GB_flag], Y_train[
TEST_BAD = getProbAccuracyScores( WHO, BAD, X_test[GB_flag], Y_test[ TARGET_F ] )

print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )
```



```

REG_GB CLASSIFICATION ACCURACY
=====
REG_GB_Train = 0.8829697986577181
REG_GB = 0.8800335570469798
-----

```

Losses

```

In [30]: AMT = LinearRegression()
          AMT = AMT.fit( W_train[GB_amt], Z_train[TARGET_A] )

          TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, W_train[GB_amt], Z_train[TAR
          TEST_AMT = getAmtAccuracyScores( WHO, AMT, W_test[GB_amt], Z_test[TARGET_A] )
          print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

          REG_GB_BAD_COEF = getCoefLogit( BAD, X_train[GB_flag] )
          REG_GB_AMT_COEF = getCoefLinear( AMT, X_train[GB_amt] )

          REG_GB_BAD = TEST_BAD.copy()
          REG_GB_AMT = TEST_AMT.copy()

```

```

REG_GB RMSE ACCURACY
=====
REG_GB_Train = 3477.5386484194455
REG_GB = 3484.9713909591555
-----

```

```

DEFAULT
-----
Total Variables: 8
INTERCEPT = -5.353184292436772
M_DEBTINC = 2.740949188158482
TRUNC_IMP_DEBTINC = 0.10690140986570486
TRUNC_IMP_DELIHQ = 0.7029918382754728
TRUNC_IMP_CLAGE = -0.006305409576988819
TRUNC_IMP_DEROG = 0.7615810886559701
M_VALUE = 3.5327054022157918
TRUNC_IMP_CLNO = -0.01604216807196335

```

```

LOSSES
-----
Total Variables: 6
INTERCEPT = -7395.372854525076
TRUNC_IMP_LOAN = 0.5160302371830359
TRUNC_IMP_CLNO = 252.31239889601392
TRUNC_IMP_DEBTINC = 158.01870021930364
M_DEBTINC = 4402.276041744145
TRUNC_IMP_CLAGE = -19.944120943066338

```

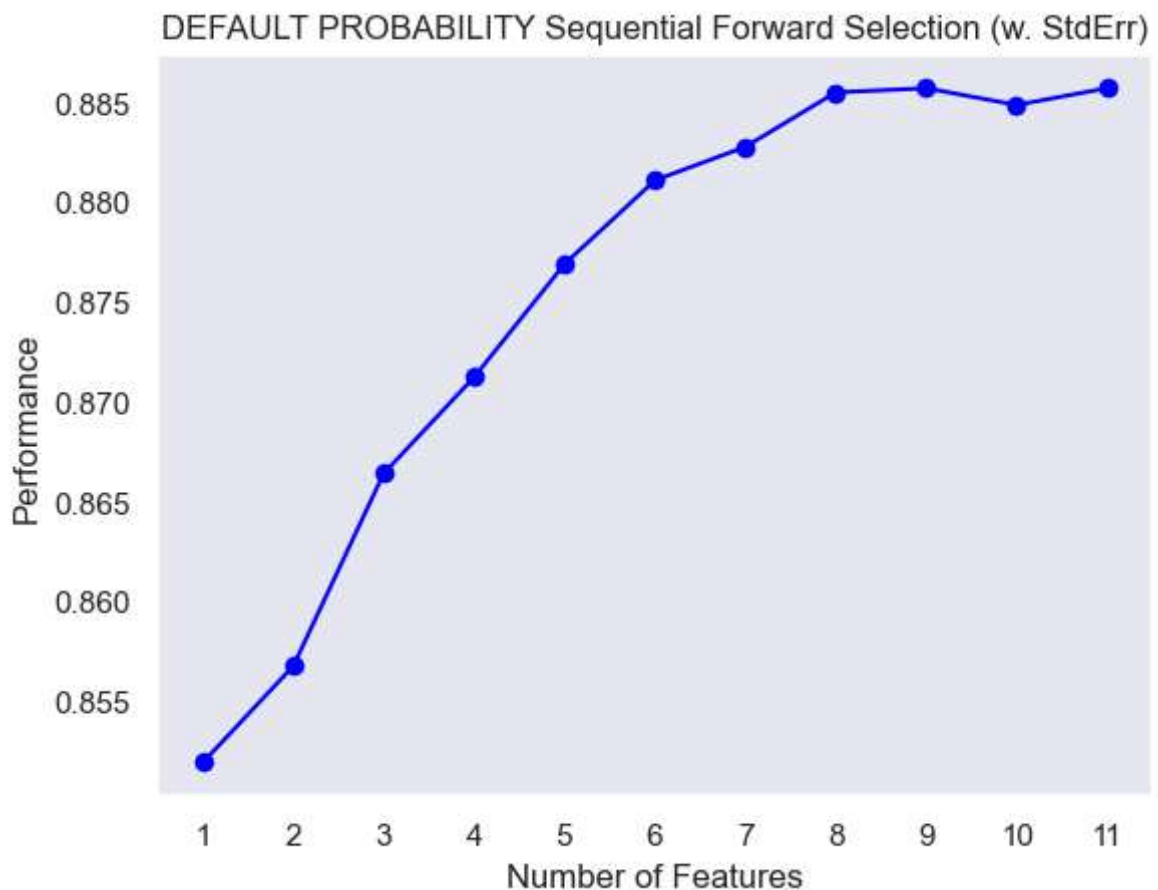
Regression: Stepwise Variable Selection


```
In [31]: U_train = X_train[ vars_tree_flag ]
stepVarNames = list( U_train.columns.values )
maxCols = U_train.shape[1]
```

```
In [32]: sfs = SFS( LogisticRegression( solver='newton-cg', max_iter=100 ),
                  k_features=( 1, maxCols ),
                  forward=True,
                  floating=False,
                  cv=3
                )
sfs.fit(U_train.values, Y_train[ TARGET_F ].values)
```

```
Out[32]: SequentialFeatureSelector ⓘ
          estimator: LogisticRegression ⓘ
              LogisticRegression ⓘ
```

```
In [33]: theFigure = plot_sfs(sfs.get_metric_dict(), kind=None )
plt.title('DEFAULT PROBABILITY Sequential Forward Selection (w. StdErr)')
plt.grid()
plt.show()
```



```
In [34]: dfm = pd.DataFrame.from_dict( sfs.get_metric_dict()).T
dfm = dfm[ ['feature_names', 'avg_score'] ]
```

```
dfm.avg_score = dfm.avg_score.astype(float)
```

```
In [35]: print(" ..... ")
maxIndex = dfm.avg_score.argmax()
print("argmax")
print( dfm.iloc[ maxIndex, ] )
print(" ..... ")

stepVars = dfm.iloc[ maxIndex, ]
stepVars = stepVars.feature_names
print( stepVars )

.....
argmax
feature_names      (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
avg_score                                0.885697
Name: 11, dtype: object
.....
('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10')
```

```
In [36]: finalStepVars = []
for i in stepVars :
    index = int(i)
    try :
        theName = stepVarNames[ index ]
        finalStepVars.append( theName )
    except :
        pass

for i in finalStepVars :
    print(i)
```

```
TRUNC_IMP_DEBTINC
M_VALUE
M_DEROG
M_DEBTINC
TRUNC_IMP_LOAN
TRUNC_IMP_VALUE
TRUNC_IMP_YOJ
TRUNC_IMP_DEROG
O_IMP_DELINQ
TRUNC_IMP_DELINQ
TRUNC_IMP_CLAGE
```

```
In [37]: U_train = X_train[ finalStepVars ]
U_test = X_test[ finalStepVars ]
```

```
In [38]: V_train = W_train[ GB_amt ]
stepVarNames = list( V_train.columns.values )
maxCols = V_train.shape[1]
```

```
In [39]: sfs = SFS( LinearRegression(),
                    k_features=( 1, maxCols ),
                    forward=True,
                    floating=False,
                    scoring = 'r2',
```

```

        cv=5
    )
sfs.fit(V_train.values, Z_train[ TARGET_A ].values)

```

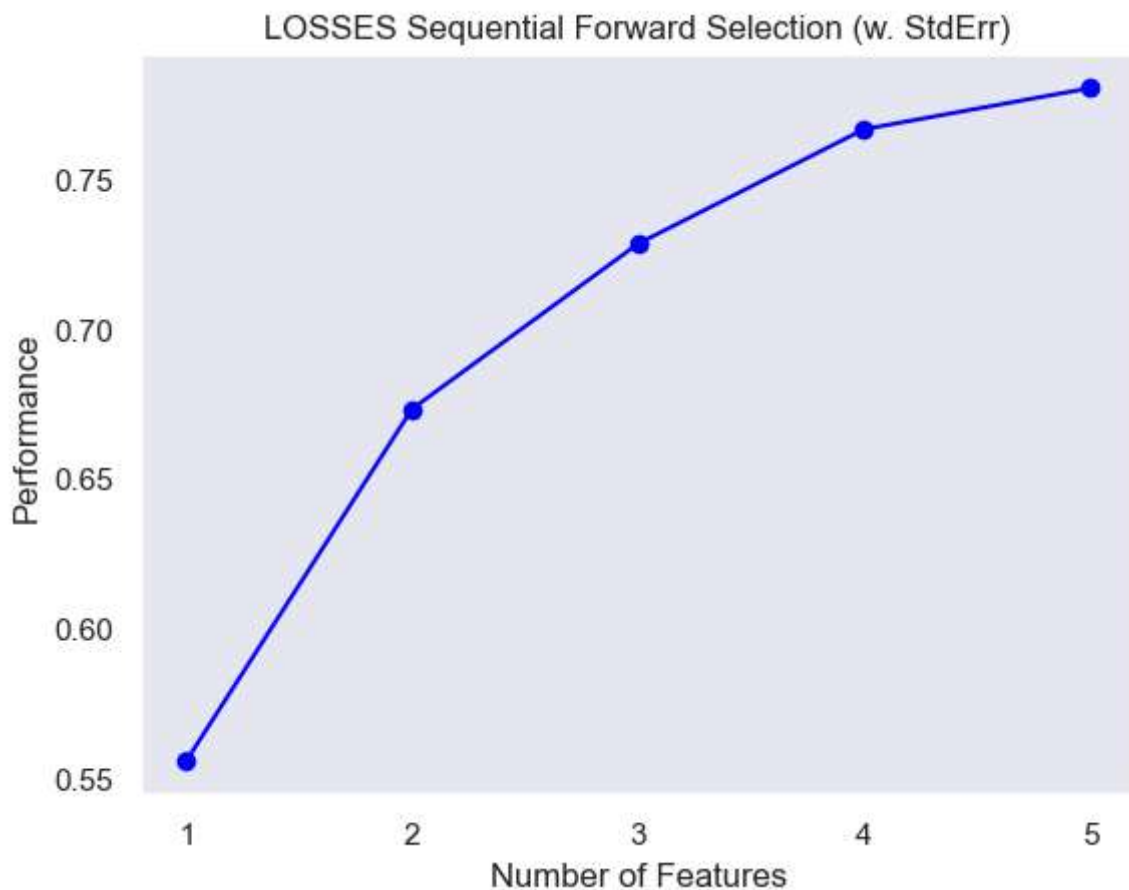
Out[39]:

- **SequentialFeatureSelector** ⓘ
- **estimator: LinearRegression**
 - **LinearRegression** ⓘ

```

In [40]: theFigure = plot_sfs(sfs.get_metric_dict(), kind=None )
plt.title('LOSSES Sequential Forward Selection (w. StdErr)')
plt.grid()
plt.show()

```



```

In [41]: dfm = pd.DataFrame.from_dict( sfs.get_metric_dict()).T
dfm = dfm[ ['feature_names', 'avg_score'] ]
dfm.avg_score = dfm.avg_score.astype(float)

```

```

In [42]: print(" ..... ")
maxIndex = dfm.avg_score.argmax()
print("argmax")
print( dfm.iloc[ maxIndex, ] )
print(" ..... ")

stepVars = dfm.iloc[ maxIndex, ]

```

```
stepVars = stepVars.feature_names
print( stepVars )
```

```
.....
argmax
feature_names      (0, 1, 2, 3, 4)
avg_score          0.78027
Name: 5, dtype: object
.....
('0', '1', '2', '3', '4')
```

```
In [43]: finalStepVars = []
        for i in stepVars :
            index = int(i)
            try :
                theName = stepVarNames[ index ]
                finalStepVars.append( theName )
            except :
                pass

        for i in finalStepVars :
            print(i)

        V_train = W_train[ finalStepVars ]
        V_test = W_test[ finalStepVars ]
```

```
TRUNC_IMP_LOAN
TRUNC_IMP_CLNO
TRUNC_IMP_DEBTINC
M_DEBTINC
TRUNC_IMP_CLAGE
```

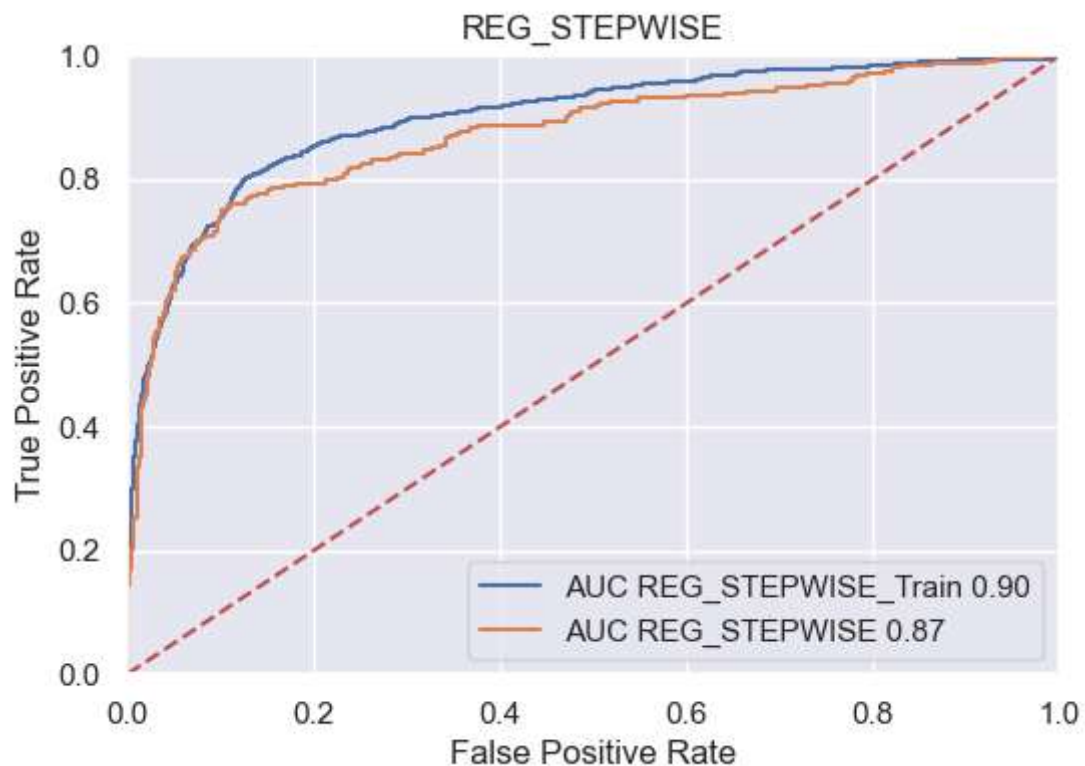
Regression

```
In [44]: WHO = "REG_STEPWISE"

        BAD = LogisticRegression( solver='newton-cg', max_iter=1000 )
        BAD = BAD.fit( U_train, Y_train[ TARGET_F ] )

        TRAIN_BAD = getProbAccuracyScores( WHO + "_Train", BAD, U_train, Y_train[ TARGET_F ]
        TEST_BAD = getProbAccuracyScores( WHO, BAD, U_test, Y_test[ TARGET_F ] )

        print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
        print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [ TRAIN_BAD, TEST_BAD ] )
```



REG_STEPWISE CLASSIFICATION ACCURACY

=====

REG_STEPWISE_Train = 0.8861157718120806

REG_STEPWISE = 0.8850671140939598

Losses

```
In [45]: AMT = LinearRegression()
AMT = AMT.fit( V_train, Z_train[TARGET_A] )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, V_train, Z_train[TARGET_A] )
TEST_AMT = getAmtAccuracyScores( WHO, AMT, V_test, Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

REG_STEP_BAD_COEF = getCoefLogit( BAD, U_train )
REG_STEP_AMT_COEF = getCoefLinear( AMT, V_train )

REG_STEP_BAD = TEST_BAD.copy()
REG_STEP_AMT = TEST_AMT.copy()
```

```

REG_STEPWISE RMSE ACCURACY
=====
REG_STEPWISE_Train = 3477.5386484194455
REG_STEPWISE = 3484.9713909591555
-----

```

```

DEFAULT
-----
Total Variables: 12
INTERCEPT = -5.2025895412174545
TRUNC_IMP_DEBTINC = 0.10465476962342855
M_VALUE = 3.7418670544564767
M_DEROG = -0.8036803571431179
M_DEBTINC = 2.7358483963235076
TRUNC_IMP_LOAN = -4.968121720548912e-06
TRUNC_IMP_VALUE = -3.800203083579819e-07
TRUNC_IMP_YOJ = -0.015280026888954145
TRUNC_IMP_DEROG = 0.6875087805092716
O_IMP_DELINQ = 2.1051181430849057
TRUNC_IMP_DELINQ = 0.6183144537388855
TRUNC_IMP_CLAGE = -0.006538797614733332

```

```

LOSSES
-----
Total Variables: 6
INTERCEPT = -7395.372854525076
TRUNC_IMP_LOAN = 0.5160302371830359
TRUNC_IMP_CLNO = 252.31239889601392
TRUNC_IMP_DEBTINC = 158.01870021930364
M_DEBTINC = 4402.276041744145
TRUNC_IMP_CLAGE = -19.944120943066338

```

```

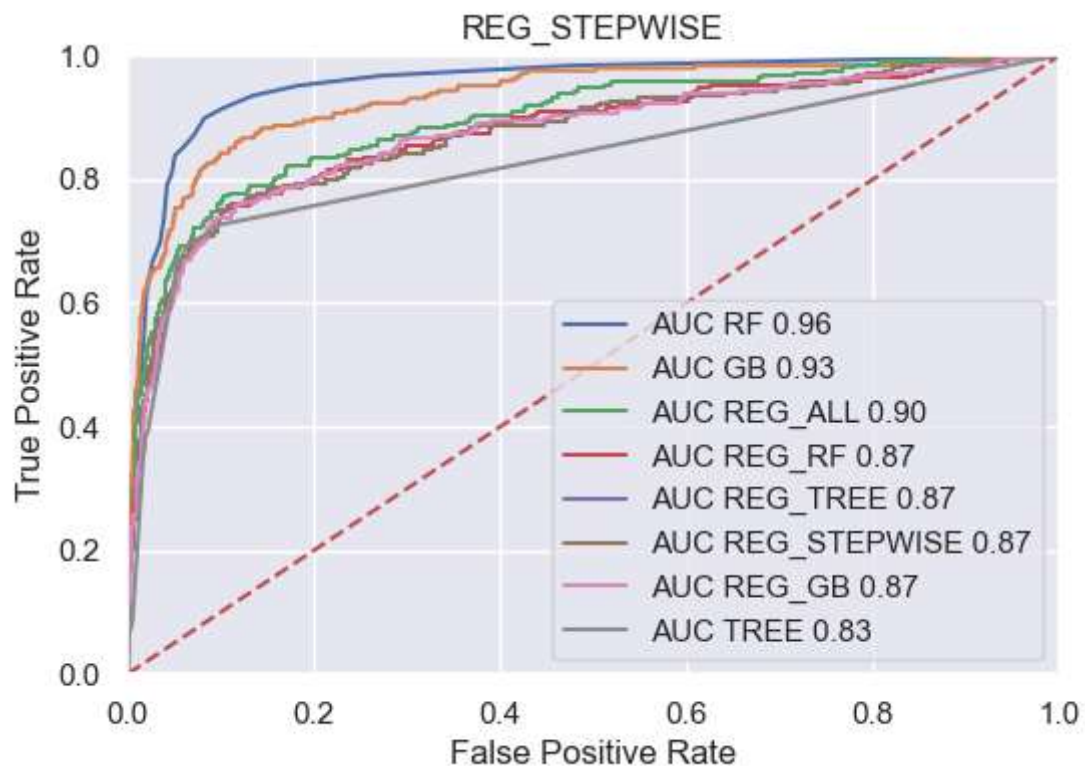
In [46]: ALL_BAD = [ TREE_BAD, RF_BAD, GB_BAD, REG_ALL_BAD, REG_TREE_BAD, REG_RF_BAD, REG_GB

ALL_BAD = sorted( ALL_BAD, key = lambda x: x[4], reverse=True )
print_ROC_Curve( WHO, ALL_BAD )

ALL_BAD = sorted( ALL_BAD, key = lambda x: x[1], reverse=True )
print_Accuracy( "ALL CLASSIFICATION ACCURACY", ALL_BAD )

ALL_AMT = [ TREE_AMT, RF_AMT, GB_AMT, REG_ALL_AMT, REG_TREE_AMT, REG_RF_AMT, REG_GB
ALL_AMT = sorted( ALL_AMT, key = lambda x: x[1] )
print_Accuracy( "ALL LOSSES MODEL ACCURACY", ALL_AMT )

```



ALL CLASSIFICATION ACCURACY

=====

RF = 0.9093959731543624
 GB = 0.9068791946308725
 TREE = 0.8875838926174496
 REG_ALL = 0.886744966442953
 REG_TREE = 0.8850671140939598
 REG_STEPWISE = 0.8850671140939598
 REG_GB = 0.8800335570469798
 REG_RF = 0.87751677852349

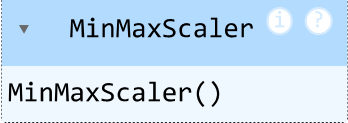
ALL LOSSES MODEL ACCURACY

=====

GB = 1872.5893967074198
 RF = 2178.4972076593153
 REG_ALL = 2936.120283227421
 REG_RF = 3484.9713909591555
 REG_GB = 3484.9713909591555
 REG_STEPWISE = 3484.9713909591555
 REG_TREE = 3680.9694661223716
 TREE = 3942.9974837311943

Tensor Flow: Predict Loan Defaults One and Two Layers (relu activation)

```
In [47]: theScaler = MinMaxScaler()
theScaler.fit(X_train)
```

```
Out[47]: 
MinMaxScaler()
```

Two Layers

```
In [48]: WHO = "Tensor_Flow"
U_train = theScaler.transform(X_train)
U_test = theScaler.transform(X_test)

U_train = pd.DataFrame(U_train)
U_test = pd.DataFrame(U_test)

U_train.columns = list(X_train.columns.values)
U_test.columns = list(X_train.columns.values)

U_train = U_train[GB_flag]
U_test = U_test[GB_flag]

F_theShapeSize = U_train.shape[1]
F_theActivation = tf.keras.activations.relu
F_theLossMetric = tf.keras.losses.SparseCategoricalCrossentropy()
F_theOptimizer = tf.keras.optimizers.Adam()
F_theEpochs = 100

F_theUnits = int(2*F_theShapeSize/3)

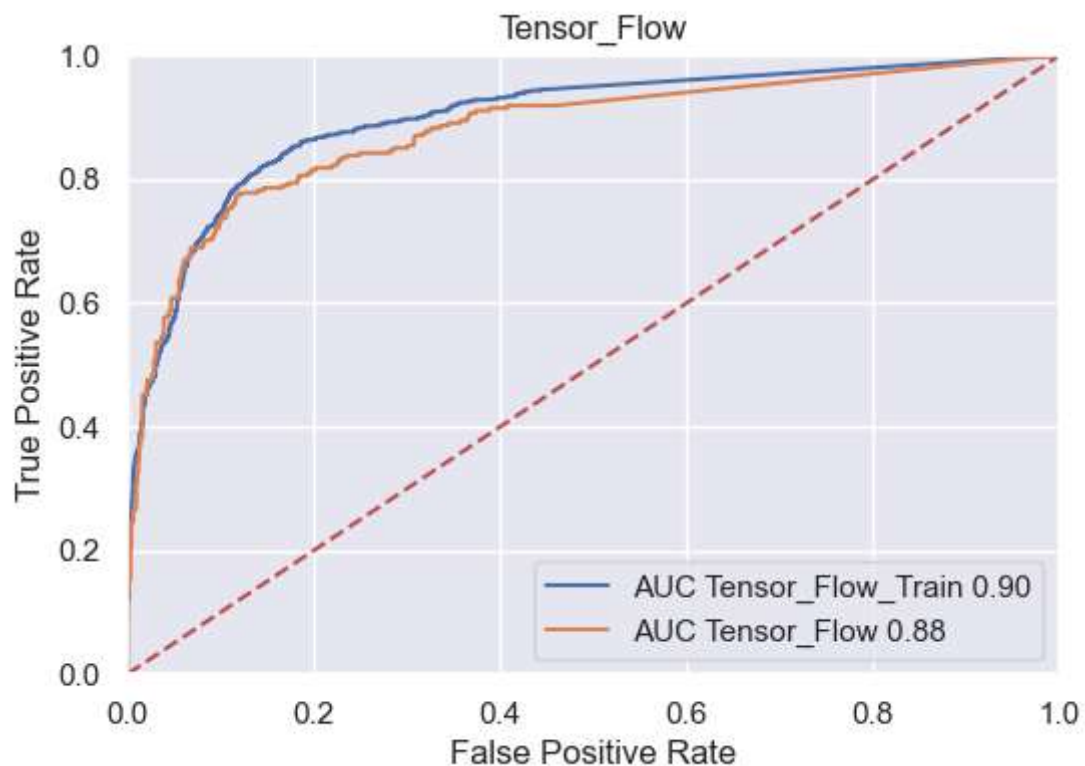
F_LAYER_01 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation,in
F_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
F_LAYER_02 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation )
F_LAYER_OUTPUT = tf.keras.layers.Dense( units=2, activation= tf.keras.activations.s

BAD = tf.keras.Sequential()
BAD.add( F_LAYER_01 )
BAD.add( F_LAYER_DROP )
BAD.add( F_LAYER_02 )
BAD.add( F_LAYER_OUTPUT )
BAD.compile( loss=F_theLossMetric, optimizer=F_theOptimizer)
BAD.fit( U_train, Y_train[TARGET_F], epochs=F_theEpochs, verbose=False )
TRAIN_BAD = get_TF_ProbAccuracyScores( WHO + "_Train", BAD, U_train,
Y_train[ TARGET_F ] )
TEST_BAD = get_TF_ProbAccuracyScores( WHO, BAD, U_test, Y_test[ TARGET_F ] )
print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [TRAIN_BAD, TEST_BAD])
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

149/149  0s 1ms/step

38/38  0s 3ms/step



```
Tensor_Flow CLASSIFICATION ACCURACY
=====
Tensor_Flow_Train = 0.8768875838926175
Tensor_Flow      = 0.8783557046979866
-----
```

One Layer

```
In [49]: U_train = theScaler.transform(X_train)
         U_test  = theScaler.transform(X_test)

         U_train = pd.DataFrame(U_train)
         U_test  = pd.DataFrame(U_test)

         U_train.columns = list(X_train.columns.values)
         U_test.columns  = list(X_train.columns.values)

         U_train = U_train[GB_flag]
         U_test  = U_test[GB_flag]

         F_theShapeSize = U_train.shape[1]
         F_theActivation = tf.keras.activations.relu
         F_theLossMetric = tf.keras.losses.SparseCategoricalCrossentropy()
         F_theOptimizer  = tf.keras.optimizers.Adam()
         F_theEpochs    = 100

         F_theUnits = int(2*F_theShapeSize/3)

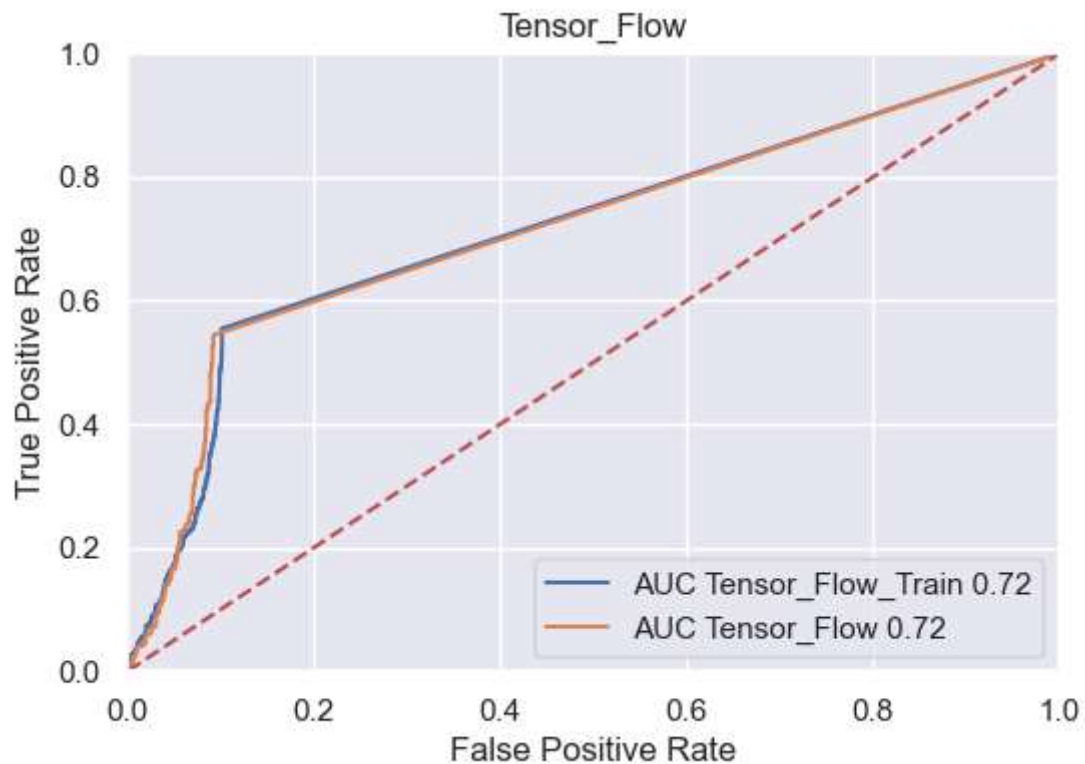
         F_LAYER_01 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation,in
         F_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
```

```

BAD = tf.keras.Sequential()
BAD.add( F_LAYER_01 )
BAD.add( F_LAYER_DROP )
BAD.compile( loss=F_theLossMetric, optimizer=F_theOptimizer)
BAD.fit( U_train, Y_train[TARGET_F], epochs=F_theEpochs, verbose=False )
TRAIN_BAD = get_TF_ProbAccuracyScores( WHO + "_Train", BAD, U_train,
Y_train[ TARGET_F ] )
TEST_BAD = get_TF_ProbAccuracyScores( WHO, BAD, U_test, Y_test[ TARGET_F ] )
print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [TRAIN_BAD, TEST_BAD])

```

149/149 ————— 0s 1ms/step
 38/38 ————— 0s 2ms/step



```

Tensor_Flow CLASSIFICATION ACCURACY
=====
Tensor_Flow_Train = 0.8263422818791947
Tensor_Flow      = 0.8204697986577181
-----

```

In [51]: `tf.config.run_functions_eagerly(True)`

Tensor Flow: Predict Loan Defaults Tanh Activation (Two Layers)

```

In [56]: F_theActivation = tf.keras.activations.tanh
F_theEpochs = 10
F_theUnits = max(8, int(2*F_theShapeSize/3))

```

```

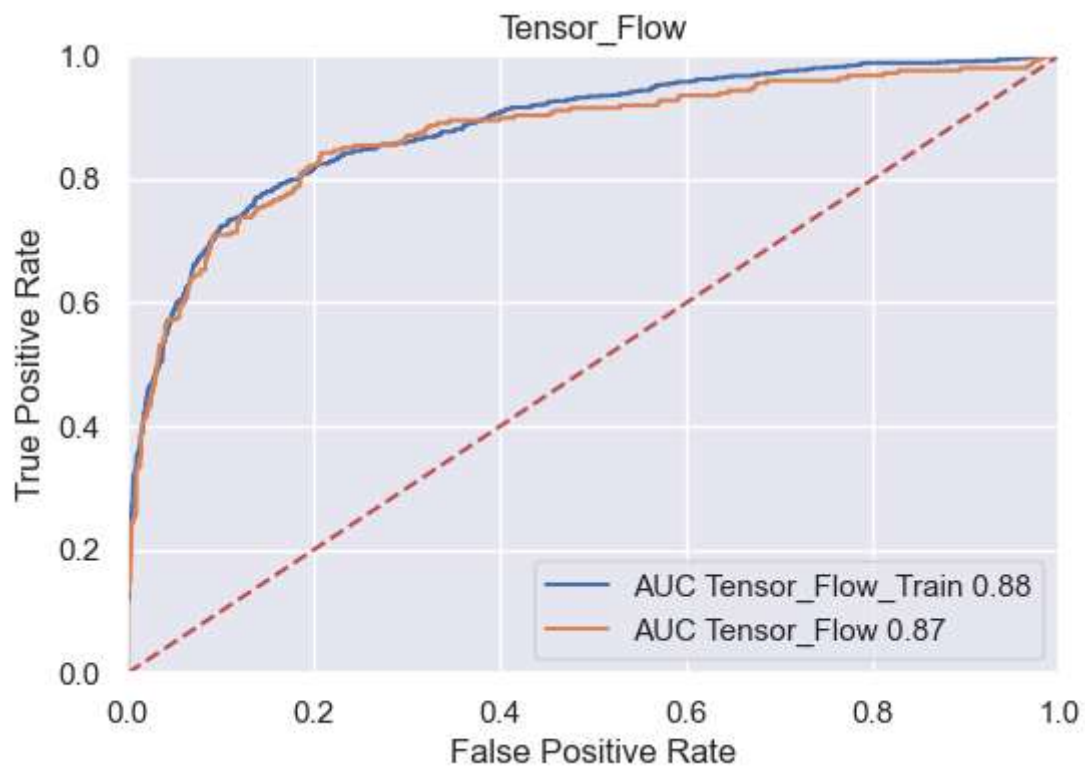
F_LAYER_01 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation,in
F_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
F_LAYER_02 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation )
F_LAYER_OUTPUT = tf.keras.layers.Dense( units=2, activation= tf.keras.activations.s

BAD = tf.keras.Sequential()
BAD.add( F_LAYER_01 )
BAD.add( F_LAYER_DROP )
BAD.add( F_LAYER_02 )
BAD.add( F_LAYER_OUTPUT )
F_theOptimizer = tf.keras.optimizers.Adam()
BAD.compile( loss=F_theLossMetric, optimizer=F_theOptimizer)
BAD.fit( U_train.values, Y_train[TARGET_F].values, epochs=F_theEpochs, verbose=False

TRAIN_BAD = get_TF_ProbAccuracyScores( WHO + "_Train", BAD, U_train,
Y_train[ TARGET_F ] )
TEST_BAD = get_TF_ProbAccuracyScores( WHO, BAD, U_test, Y_test[ TARGET_F ] )
print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [TRAIN_BAD, TEST_BAD])

```

149/149 ————— 1s 6ms/step
38/38 ————— 0s 6ms/step



```

Tensor_Flow CLASSIFICATION ACCURACY
=====
Tensor_Flow_Train = 0.8783557046979866
Tensor_Flow      = 0.87248322147651
-----

```

Tensor Flow: Predict Loan Defaults ELU Activation (Two Layers)

```

In [58]: F_theActivation = tf.keras.activations.elu
F_theEpochs = 10
F_theUnits = max(8, int(2*F_theShapeSize/3))

F_LAYER_01 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation,in
F_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
F_LAYER_02 = tf.keras.layers.Dense( units=F_theUnits, activation=F_theActivation )
F_LAYER_OUTPUT = tf.keras.layers.Dense( units=2, activation= tf.keras.activations.s

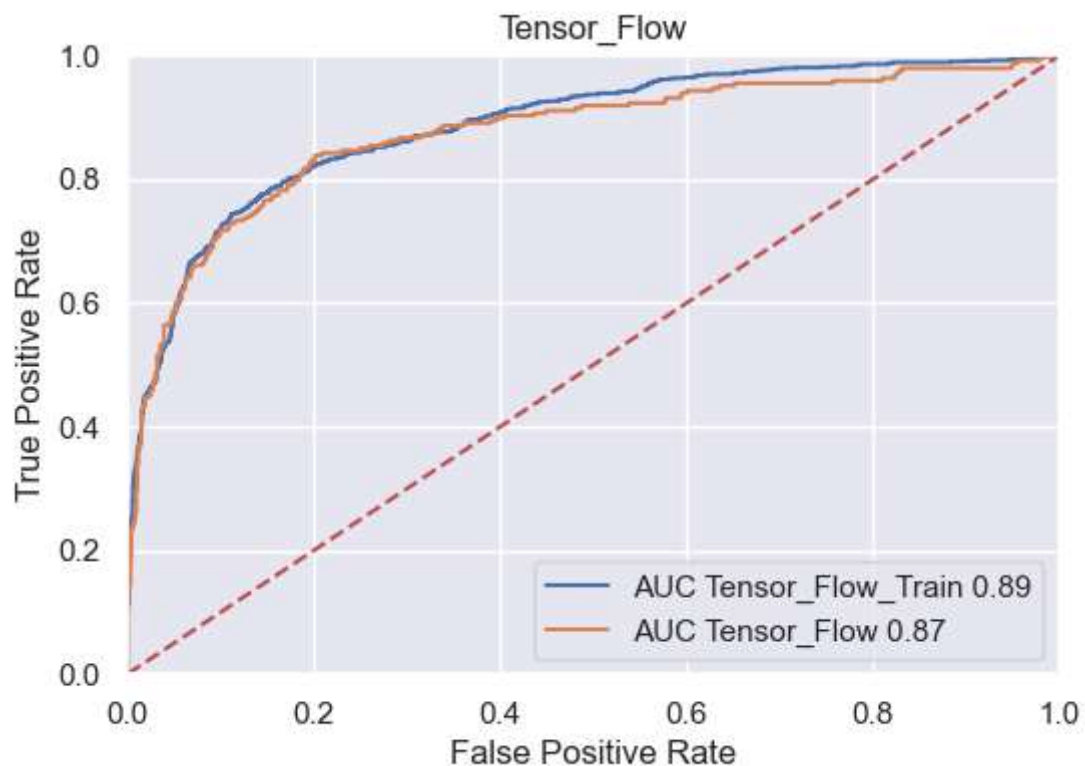
BAD = tf.keras.Sequential()
BAD.add( F_LAYER_01 )
BAD.add( F_LAYER_DROP )
BAD.add( F_LAYER_02 )
BAD.add( F_LAYER_OUTPUT )
F_theOptimizer = tf.keras.optimizers.Adam()
BAD.compile( loss=F_theLossMetric, optimizer=F_theOptimizer)
BAD.fit( U_train.values, Y_train[TARGET_F].values, epochs=F_theEpochs, verbose=False

TRAIN_BAD = get_TF_ProbAccuracyScores( WHO + "_Train", BAD, U_train,
Y_train[ TARGET_F ] )
TEST_BAD = get_TF_ProbAccuracyScores( WHO, BAD, U_test, Y_test[ TARGET_F ] )
print_ROC_Curve( WHO, [ TRAIN_BAD, TEST_BAD ] )
print_Accuracy( WHO + " CLASSIFICATION ACCURACY", [TRAIN_BAD, TEST_BAD])

```

149/149 ————— 1s 5ms/step

38/38 ————— 0s 6ms/step



```

Tensor_Flow CLASSIFICATION ACCURACY
=====
Tensor_Flow_Train = 0.875
Tensor_Flow = 0.8758389261744967
-----

```

TensorFlow: Predict Loss Given Default One and Two Layers (ReLU activation)

Two Layers

```

In [60]: V_train = theScaler.transform( W_train )
V_test = theScaler.transform( W_test )
V_train = pd.DataFrame( V_train )
V_test = pd.DataFrame( V_test )
V_train.columns = list( W_train.columns.values )
V_test.columns = list(W_train.columns.values)
V_train = V_train[GB_amt]
V_test = V_test[GB_amt]

A_theShapeSize = V_train.shape[1]
A_theActivation = tf.keras.activations.relu
A_theLossMetric = tf.keras.losses.MeanSquaredError()
A_theOptimizer = tf.keras.optimizers.Adam()
A_theEpochs = 100

A_theUnits = max(8, int(2*F_theShapeSize/3))

A_LAYER_01 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation,in
A_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
A_LAYER_02 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation )
A_LAYER_OUTPUT = tf.keras.layers.Dense( units=1,activation=tf.keras.activations.lin

AMT = tf.keras.Sequential()
AMT.add( A_LAYER_01 )
AMT.add( A_LAYER_DROP )
AMT.add( A_LAYER_02 )
AMT.add( A_LAYER_OUTPUT )
AMT.compile( loss=A_theLossMetric, optimizer=A_theOptimizer)
AMT.fit( V_train, Z_train[TARGET_A], epochs=A_theEpochs, verbose=False )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, V_train[GB_amt], Z_train[TAR
TEST_AMT = getAmtAccuracyScores( WHO, AMT, V_test[GB_amt], Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

TF_BAD = TEST_BAD.copy()
TF_AMT = TEST_AMT.copy()

ALL_BAD = [ TREE_BAD, RF_BAD, GB_BAD, REG_ALL_BAD, REG_TREE_BAD, REG_RF_BAD,REG_GB_
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[4], reverse=True )
print_ROC_Curve( WHO, ALL_BAD )

```

```

ALL_BAD = sorted( ALL_BAD, key = lambda x: x[1], reverse=True )
print_Accuracy( "ALL CLASSIFICATION ACCURACY", ALL_BAD )
ALL_AMT = [ TREE_AMT, RF_AMT, GB_AMT, REG_ALL_AMT, REG_TREE_AMT, REG_RF_AMT, REG_GB_
ALL_AMT = sorted( ALL_AMT, key = lambda x: x[1] )
print_Accuracy( "ALL DAMAGE MODEL ACCURACY", ALL_AMT)

```

30/30 ————— 0s 6ms/step

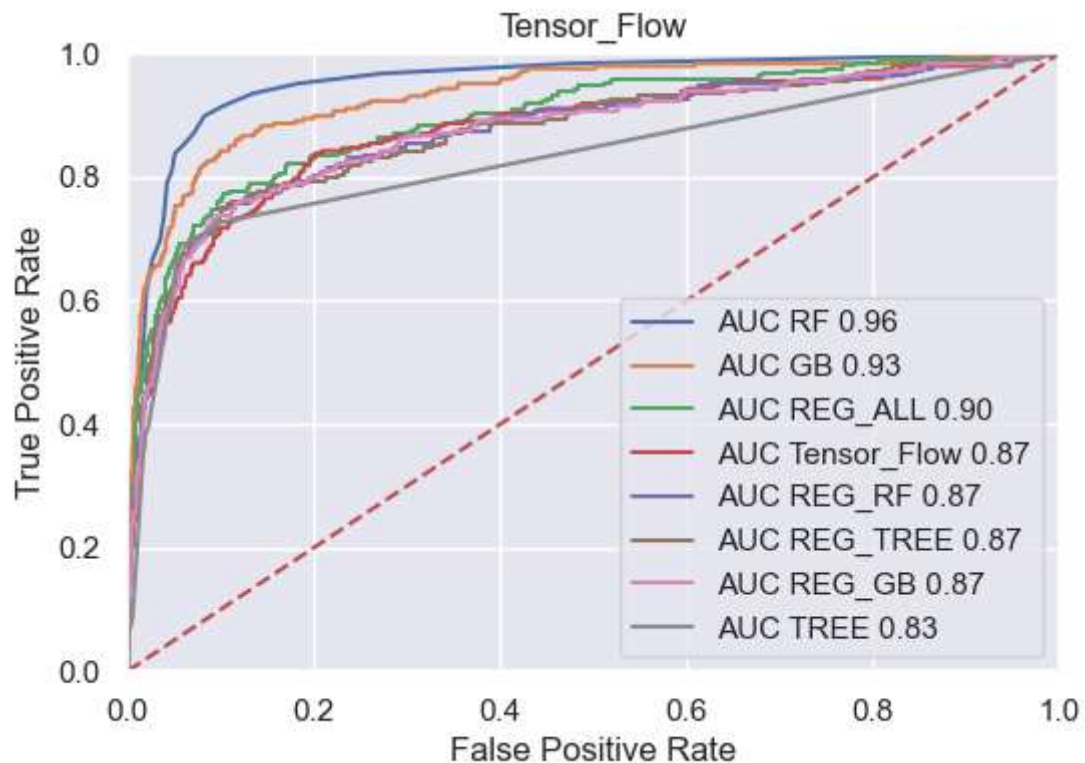
8/8 ————— 0s 8ms/step

Tensor_Flow RMSE ACCURACY

=====

Tensor_Flow_Train = 7428.265420059634

Tensor_Flow = 7335.6316222938585



```

ALL CLASSIFICATION ACCURACY
=====
RF = 0.9093959731543624
GB = 0.9068791946308725
TREE = 0.8875838926174496
REG_ALL = 0.886744966442953
REG_TREE = 0.8850671140939598
REG_GB = 0.8800335570469798
REG_RF = 0.87751677852349
Tensor_Flow = 0.8758389261744967
-----

```

```

ALL DAMAGE MODEL ACCURACY
=====
GB = 1872.5893967074198
RF = 2178.4972076593153
REG_ALL = 2936.120283227421
REG_RF = 3484.9713909591555
REG_GB = 3484.9713909591555
REG_TREE = 3680.9694661223716
TREE = 3942.9974837311943
Tensor_Flow = 7335.6316222938585
-----

```

One Layer

```

In [61]: V_train = theScaler.transform( W_train )
V_test = theScaler.transform( W_test )
V_train = pd.DataFrame( V_train )
V_test = pd.DataFrame( V_test )
V_train.columns = list( W_train.columns.values )
V_test.columns = list(W_train.columns.values)
V_train = V_train[GB_amt]
V_test = V_test[GB_amt]

A_theShapeSize = V_train.shape[1]
A_theActivation = tf.keras.activations.relu
A_theLossMetric = tf.keras.losses.MeanSquaredError()
A_theOptimizer = tf.keras.optimizers.Adam()
A_theEpochs = 100

A_theUnits = max(8, int(2*F_theShapeSize/3))

A_LAYER_01 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation,in
A_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
A_LAYER_OUTPUT = tf.keras.layers.Dense(units=1, activation="linear")

AMT = tf.keras.Sequential()
AMT.add( A_LAYER_01 )
AMT.add( A_LAYER_DROP )
AMT.add( A_LAYER_OUTPUT )
AMT.compile( loss=A_theLossMetric, optimizer=A_theOptimizer)

```



```

AMT.fit( V_train, Z_train[TARGET_A], epochs=A_theEpochs, verbose=False )

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, V_train[GB_amt], Z_train[TAR
TEST_AMT = getAmtAccuracyScores( WHO, AMT, V_test[GB_amt], Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

TF_BAD = TEST_BAD.copy()
TF_AMT = TEST_AMT.copy()

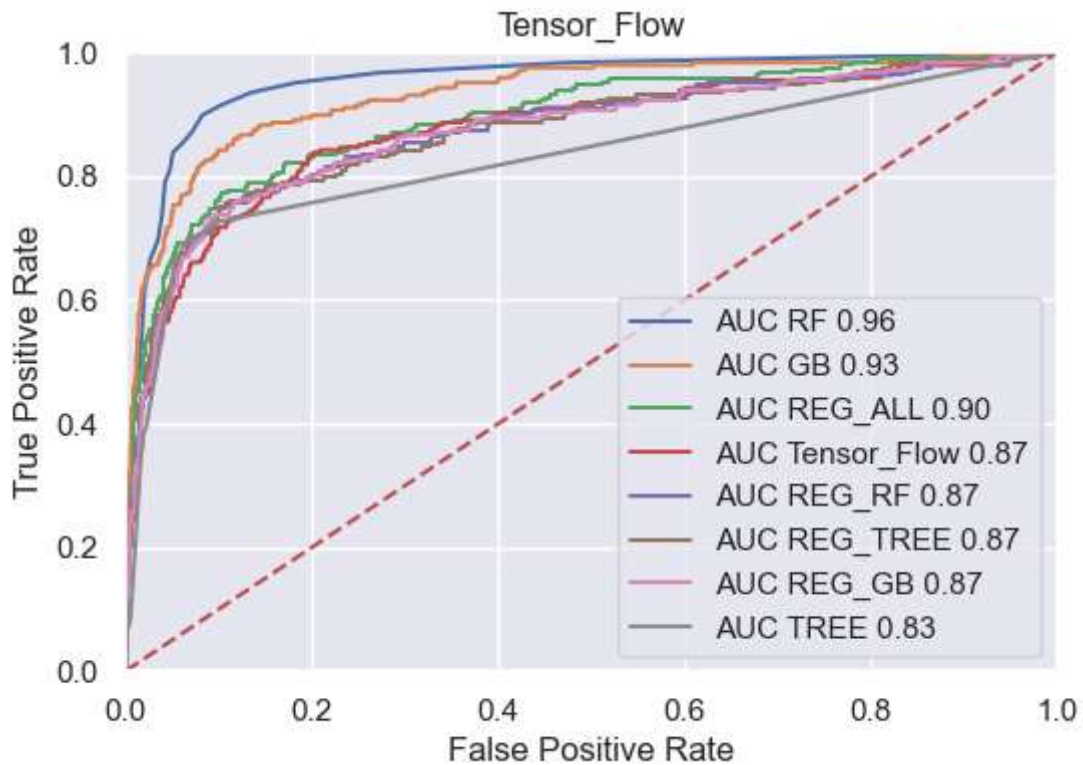
ALL_BAD = [ TREE_BAD, RF_BAD, GB_BAD, REG_ALL_BAD, REG_TREE_BAD, REG_RF_BAD,REG_GB_
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[4], reverse=True )
print_ROC_Curve( WHO, ALL_BAD )
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[1], reverse=True )
print_Accuracy( "ALL CLASSIFICATION ACCURACY", ALL_BAD )
ALL_AMT = [ TREE_AMT, RF_AMT, GB_AMT, REG_ALL_AMT, REG_TREE_AMT, REG_RF_AMT,REG_GB_
ALL_AMT = sorted( ALL_AMT, key = lambda x: x[1] )
print_Accuracy( "ALL DAMAGE MODEL ACCURACY", ALL_AMT)

```

```

30/30 ————— 0s 4ms/step
8/8 ————— 0s 8ms/step
Tensor_Flow RMSE ACCURACY
=====
Tensor_Flow_Train = 14216.706173736598
Tensor_Flow = 14010.24267485516
-----

```




```

ALL CLASSIFICATION ACCURACY
=====
RF = 0.9093959731543624
GB = 0.9068791946308725
TREE = 0.8875838926174496
REG_ALL = 0.886744966442953
REG_TREE = 0.8850671140939598
REG_GB = 0.8800335570469798
REG_RF = 0.87751677852349
Tensor_Flow = 0.8758389261744967
-----

```

```

ALL DAMAGE MODEL ACCURACY
=====
GB = 1872.5893967074198
RF = 2178.4972076593153
REG_ALL = 2936.120283227421
REG_RF = 3484.9713909591555
REG_GB = 3484.9713909591555
REG_TREE = 3680.9694661223716
TREE = 3942.9974837311943
Tensor_Flow = 14010.24267485516
-----

```

Tensor Flow: Predict Loss Given Default Tanh Activation (Two Layers)

```
In [62]: tf.config.run_functions_eagerly(True)
```

```
In [63]: A_theActivation = tf.keras.activations.tanh

A_LAYER_01 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation,in
A_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
A_LAYER_02 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation )
A_LAYER_OUTPUT = tf.keras.layers.Dense( units=1,activation=tf.keras.activations.lin

AMT = tf.keras.Sequential()
AMT.add( A_LAYER_01 )
AMT.add( A_LAYER_DROP )
AMT.add( A_LAYER_02 )
AMT.add( A_LAYER_OUTPUT )
A_theOptimizer = tf.keras.optimizers.Adam()
AMT.compile( loss=A_theLossMetric, optimizer=A_theOptimizer)
AMT.fit(V_train.values, Z_train[TARGET_A].values, epochs=A_theEpochs, verbose=False

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, V_train[GB_amt], Z_train[TAR
TEST_AMT = getAmtAccuracyScores( WHO, AMT, V_test[GB_amt], Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

TF_BAD = TEST_BAD.copy()
TF_AMT = TEST_AMT.copy()
```

```

ALL_BAD = [ TREE_BAD, RF_BAD, GB_BAD, REG_ALL_BAD, REG_TREE_BAD, REG_RF_BAD, REG_GB_
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[4], reverse=True )
print_ROC_Curve( WHO, ALL_BAD )
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[1], reverse=True )
print_Accuracy( "ALL CLASSIFICATION ACCURACY", ALL_BAD )
ALL_AMT = [ TREE_AMT, RF_AMT, GB_AMT, REG_ALL_AMT, REG_TREE_AMT, REG_RF_AMT, REG_GB_
ALL_AMT = sorted( ALL_AMT, key = lambda x: x[1] )
print_Accuracy( "ALL DAMAGE MODEL ACCURACY", ALL_AMT)

```

30/30 ————— 0s 5ms/step

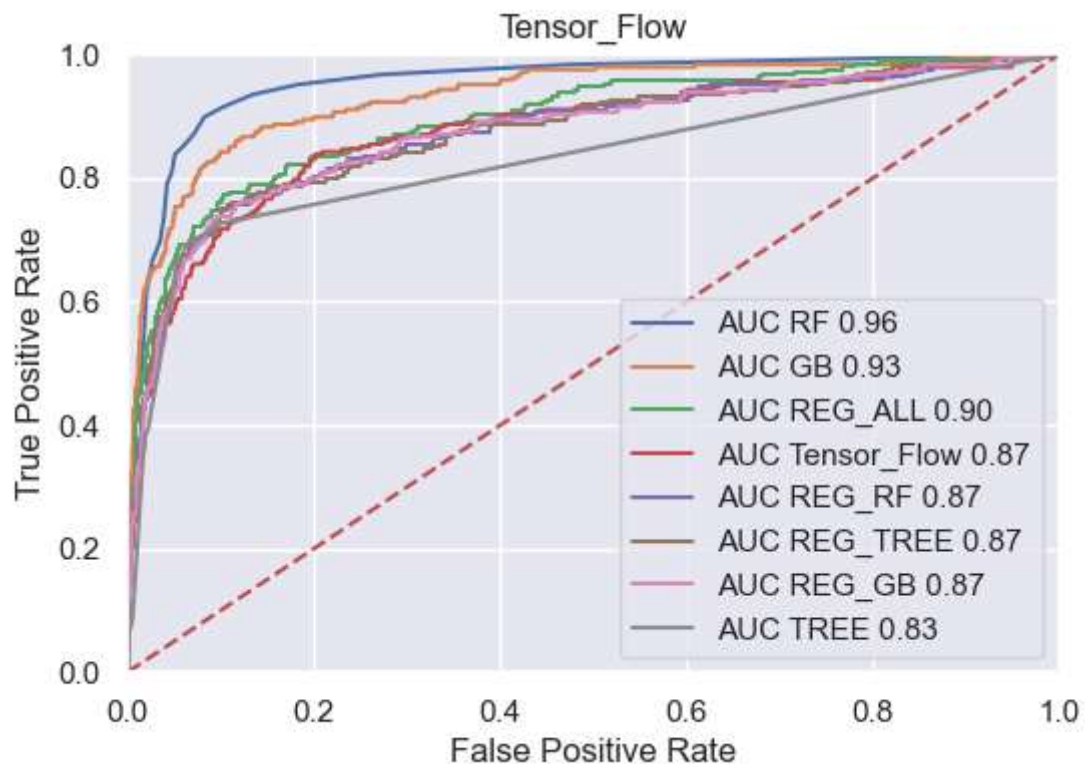
8/8 ————— 0s 8ms/step

Tensor_Flow RMSE ACCURACY

=====

Tensor_Flow_Train = 14304.88788580904

Tensor_Flow = 14097.043493667965



```

ALL CLASSIFICATION ACCURACY
=====
RF = 0.9093959731543624
GB = 0.9068791946308725
TREE = 0.8875838926174496
REG_ALL = 0.886744966442953
REG_TREE = 0.8850671140939598
REG_GB = 0.8800335570469798
REG_RF = 0.87751677852349
Tensor_Flow = 0.8758389261744967
-----

```

```

ALL DAMAGE MODEL ACCURACY
=====
GB = 1872.5893967074198
RF = 2178.4972076593153
REG_ALL = 2936.120283227421
REG_RF = 3484.9713909591555
REG_GB = 3484.9713909591555
REG_TREE = 3680.9694661223716
TREE = 3942.9974837311943
Tensor_Flow = 14097.043493667965
-----

```

Tensor Flow: Predict Loss Given Default ELU Activation (Two Layers)

```

In [65]: A_theActivation = tf.keras.activations.elu

A_LAYER_01 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation,in
A_LAYER_DROP = tf.keras.layers.Dropout( 0.2 )
A_LAYER_02 = tf.keras.layers.Dense( units=A_theUnits, activation=A_theActivation )
A_LAYER_OUTPUT = tf.keras.layers.Dense( units=1,activation=tf.keras.activations.lin

AMT = tf.keras.Sequential()
AMT.add( A_LAYER_01 )
AMT.add( A_LAYER_DROP )
AMT.add( A_LAYER_02 )
AMT.add( A_LAYER_OUTPUT )
A_theOptimizer = tf.keras.optimizers.Adam()
AMT.compile( loss=A_theLossMetric, optimizer=A_theOptimizer)
AMT.fit( V_train.values, Z_train[TARGET_A].values, epochs=A_theEpochs, verbose=False

TRAIN_AMT = getAmtAccuracyScores( WHO + "_Train", AMT, V_train[GB_amt],
Z_train[TARGET_A] )
TEST_AMT = getAmtAccuracyScores( WHO, AMT, V_test[GB_amt], Z_test[TARGET_A] )
print_Accuracy( WHO + " RMSE ACCURACY", [ TRAIN_AMT, TEST_AMT ] )

TF_BAD = TEST_BAD.copy()
TF_AMT = TEST_AMT.copy()

```

```

ALL_BAD = [ TREE_BAD, RF_BAD, GB_BAD, REG_ALL_BAD, REG_TREE_BAD, REG_RF_BAD, REG_GB_
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[4], reverse=True )
print_ROC_Curve( WHO, ALL_BAD )
ALL_BAD = sorted( ALL_BAD, key = lambda x: x[1], reverse=True )
print_Accuracy( "ALL CLASSIFICATION ACCURACY", ALL_BAD )
ALL_AMT = [ TREE_AMT, RF_AMT, GB_AMT, REG_ALL_AMT, REG_TREE_AMT, REG_RF_AMT, REG_GB_
ALL_AMT = sorted( ALL_AMT, key = lambda x: x[1] )
print_Accuracy( "ALL DAMAGE MODEL ACCURACY", ALL_AMT)

```

30/30 ————— 0s 6ms/step

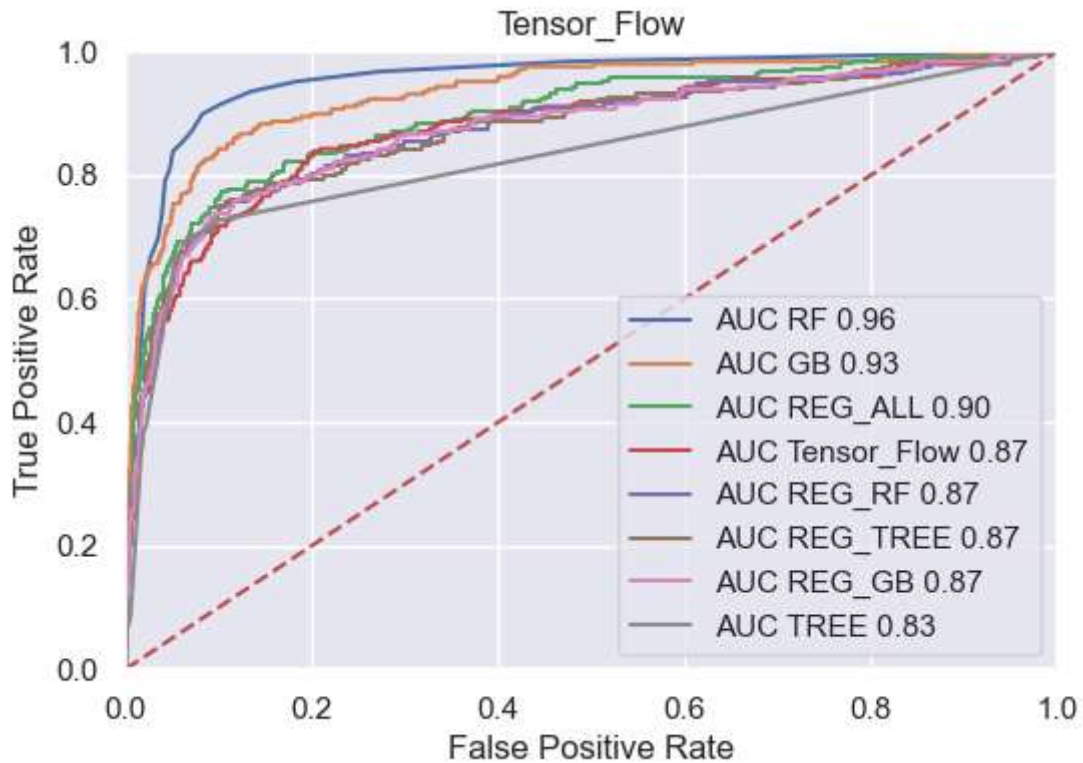
8/8 ————— 0s 8ms/step

Tensor_Flow RMSE ACCURACY

=====

Tensor_Flow_Train = 10839.63801728372

Tensor_Flow = 10683.384753195254



```
ALL CLASSIFICATION ACCURACY
=====
RF = 0.9093959731543624
GB = 0.9068791946308725
TREE = 0.8875838926174496
REG_ALL = 0.886744966442953
REG_TREE = 0.8850671140939598
REG_GB = 0.8800335570469798
REG_RF = 0.87751677852349
Tensor_Flow = 0.8758389261744967
-----
```

```
ALL DAMAGE MODEL ACCURACY
=====
GB = 1872.5893967074198
RF = 2178.4972076593153
REG_ALL = 2936.120283227421
REG_RF = 3484.9713909591555
REG_GB = 3484.9713909591555
REG_TREE = 3680.9694661223716
TREE = 3942.9974837311943
Tensor_Flow = 10683.384753195254
-----
```

In []: